

Computer System Laboratory テクニカルハンドアウト (Handout)

- 授業名: 「Computer System Laboratory」全8回シリーズ
- 対象読者: 受講者全員（講義中および90分演習時の手元参照用リファレンス）
- 時間設定: 各回 講義 60分 + 演習 90分

1. 信号型・演算子クイックリファレンス (第1回～第2回)

構文	意味	物理回路	記述例
<code>Bool()</code>	1ビット二値信号	1本の配線	<code>val en = Input(Bool())</code>
<code>UInt(w.W)</code>	w ビット符号なし整数	w 本のバス	<code>val data = Wire(UInt(8.W))</code>
<code>SInt(w.W)</code>	w ビット2の補数整数	w 本のバス	<code>val s = Wire(SInt(16.W))</code>
<code>+&</code>	拡張加算 (キャリー保持)	$w + 1$ ビット加算器	<code>val sumExt = a +& b</code>
<code>-&</code>	拡張減算 (ボロー保持)	$w + 1$ ビット減算器	<code>val subExt = a -& b</code>
<code>Cat(hi, lo)</code>	バス結合 (上位/下位)	物理配線バンドル	<code>val word = Cat(highNibble, lowNibble)</code>
<code>x(hi, lo)</code>	ビットスライス抽出	バスから部分配線	<code>val slice = x(7, 4)</code>
<code>x.orR</code>	縮約OR (Reduction OR)	ORツリー回路	<code>val anyBit = x.orR</code> (非ゼロ判定)
<code>Mux(c, t, f)</code>	2-to-1 マルチプレクサ	物理MUXゲート	<code>val res = Mux(sel, in1, in0)</code>
<code>WireDefault(v)</code>	デフォルト接続付き配線	透過ラッチ完全防止	<code>val out = WireDefault(0.U(8.W))</code>
<code>switch / is</code>	直交デコーダ	並列平衡デコーダ	<code>switch(op) { is(0.U) { ... } }</code>

2. 順序回路と特殊カウンタ構文 (第3回～第4回)

回路要素	Chisel記述	ハードウェア動作特性
初期値付きD-FF	<code>val r = RegInit(0.U(8.W))</code>	暗黙の同期リセットで初期化
パイプライン段	<code>val pipe = RegNext(inSignal, 0.U)</code>	1クロックサイクルの同期遅延
モジュロNラップ	<code>when(cnt === (n-1).U) { cnt := 0.U }</code>	終端検出による巡回リセット
Ringカウンタ循環	<code>Cat(state(w - 2, 0), state(w - 1))</code>	1-hot左循環シフト
Johnson反転帰還	<code>Cat(state(w - 2, 0), ~state(w - 1))</code>	最上位ビットの反転を最下位へ帰還 ($2N$ 状態)
1-hot自己復帰判定	<code>PopCount(state) === 1.U</code>	立っているビット数が1以外の時に初期値へ強制復帰
グレイコード変換	<code>val gray = bin ^ (bin >> 1)</code>	隣接遷移で常に1ビットのみ反転する安全符号

3. 構造化設計と同期カスケード (第5回～第6回)

サブモジュールのインスタンス化と配線

```
class TopModule extends Module {  
  val subA = Module(new SubModuleA)  
  val subB = Module(new SubModuleB)  
  
  // 1. 親の入力 -> 子の入力  
  subA.io.in := io.topIn  
  
  // 2. 子 -> 子 の完全同期カスケード (cout -> en)  
  subB.io.en := io.en && subA.io.rollover  
  
  // 3. 子の出力 -> 親の出力  
  io.topOut := subB.io.out  
}
```

同期カスケードの絶対原則

- **非同期リップルクロックの厳禁:** 下位モジュールの `cout` を上位モジュールの `clock` 端子に接続してはならない。
- すべてのモジュールに同一のシステムクロックを供給し、`cout` を上位の `en` 端子に接続する。

4. FSMとメモリ回路の構文 (第7回～第8回)

有限状態機械 (ChiselEnum + Moore型)

```
object State extends ChiselEnum {  
  val sIDLE, sACTIVE, sDONE = Value  
}  
  
val stateReg = RegInit(State.sIDLE)  
  
// 次状態遷移論理  
switch(stateReg) {  
  is(State.sIDLE) { when(io.start) { stateReg := State.sACTIVE } }  
  is(State.sACTIVE) { when(io.finish) { stateReg := State.sDONE } }  
  is(State.sDONE) { stateReg := State.sIDLE }  
}  
  
// Moore型出力 (状態レジスタのみから生成 = グリッチフリー)  
io.busy := stateReg === State.sACTIVE
```

非同期読み出しメモリ & RAWフォワーディング (Mem)

```
val mem = Mem(8, UInt(8.W))  
  
// クロック同期書き込み  
when(io.wen) { mem(io.waddr) := io.wdata }  
  
// 非同期読み出し (同一サイクル無遅延)  
val rawData = mem(io.raddr)  
  
// RAWフォワーディング回路 (書き込み中の最新データを即時バイパス)  
val bypass = io.wen && (io.waddr === io.raddr)  
io.rdata := Mux(bypass, io.wdata, rawData)
```

5. 演習課題一覧とチェックリスト (90分演習)

回	基本問題（必須 / 45分）	発展問題（推奨 / 35分）	重点チェック項目
第1回	FullAdder , RippleCarryAdder4	AddSub4 (減算・オーバーフロー)	拡張加算 <code>+&</code> 、符号反転 $B \oplus sub$
第2回	Mux4to1 , SevenSegDecoder	PriorityEncoder4 (valid 出力)	WireDefault、 <code>io.in.orR</code>
第3回	ModuloNCounter (クリア/ロード/en)	PwmGenerator (周期/Duty比較)	$N - 1$ ラップ、優先度 <code>clear > load > en</code>
第4回	SelfCorrectingRingCounter , JohnsonCounter	GrayCodeCounter (ハミング距離1)	<code>PopCount(state) === 1.U</code> 自己復帰
第5回	StructuredALU (算術/論理統合)	PipelinedALUWithFlags (パイプライン)	<code>Module(new ...)</code> 、 <code>RegNext</code> 遅延同期
第6回	Counter60 (10進×6進カスケード)	DigitalClockCore (24時間時計)	リップルクロック厳禁、 <code>cout</code> → <code>en</code>
第7回	SequenceDetector101 (Moore型)	TrafficLightController (タイマ連動)	<code>ChiselEnum</code> 、重複 "10101" 検出
第8回	RegisterFile8x8 (2R1W 非同期)	BypassedRegisterFile (RAWバイパス)	同一サイクルでの <code>waddr === raddr</code> バイパス